Introdução Lab Docker

Comandos básicos

```
docker --version
```

```
docker run hello-world
```

```
docker run ubuntu
```

```
docker run -it ubuntu
```

```
apt update && apt install python3


python3 -V
```

Stateless

```
docker run -it ubuntu
python3 -V
```

```
docker run -it ubuntu
rm -rf / # don't run it on your computer!
```

Containers

```
docker ps -a
```

```
docker rm $(docker ps -aq)
```

```
docker run -it --rm ubuntu
```

Diferentes imagens base

```
docker run -it --rm python:3.9.16
# add -slim to get a smaller version
```

```
docker run -it --rm python:3.9.16-slim
```

```
docker run -it --rm --entrypoint=bash python:3.9.16-slim
```

Volumes

```
mkdir teste_volume
cd teste_volume
Set-Content -Path "arquivo1.txt" -Value "Hello from host"
Set-Content -Path "arquivo2.txt" -Value "sou um arquivo do host"
Set-Content -Path "arquivo3.txt" -Value "arquivo3 sou um arquivo do host"
cd ..
```

```
from pathlib import Path

current_dir = Path.cwd()
current_file = Path(__file__).name

print(f"Files in {current_dir}:")

for filepath in current_dir.iterdir():
    if filepath.name == current_file:
        continue

    print(f"  - {filepath.name}")

    if filepath.is_file():
        content = filepath.read_text(encoding='utf-8')
        print(f"    Content: {content}")
```

```
docker run -it  --rm  --volume $pwd/teste_volume:/app/test/  --entrypoint=bash  python:3.9
```

Dentro do container

```
cd /app/test
```

```
cat arquivo1.txt.txt
python script.py
```

Script que recebe argumentos

```
import sys

print("Argumentos de linha de comando:", sys.argv)

arg1 = sys.argv[1] if len(sys.argv) > 1 else "Valor padrão para arg1"
arg2 = sys.argv[2] if len(sys.argv) > 2 else "Valor padrão para arg2"

print(f"Argumento 1: {arg1}")
print(f"Argumento 2: {arg2}")
```

Trabalhar com ambiente virtual

```
pip install uv
```

```
-->
```

```
uv init --python 3.14
```

```
uv add pandas pyarrow


OU

uv add pandas pyarrow --link-mode=copy
```

```
uv run python .\pipeline.py Sudeste Nordeste
```

Dockerizing the Pipeline

Dockerfile com pip

```
# base Docker image that we will build on
FROM python:3.14.3-slim
```

```
# set up our image by installing prerequisites; pandas in this case
RUN pip install pandas pyarrow

# set up the working directory inside the container
WORKDIR /app
# copy the script to the container. 1st name is source file, 2nd is destination
COPY pipeline.py pipeline.py

# define what to do first when the container runs
# in this example, we will just run the script
ENTRYPOINT ["python", "pipeline.py"]
```

Construir e rodar imagem

```
docker build -t test:pipeline .
```

```
docker run  test:pipeline
```

```
docker run -it --entrypoint bash --rm test:pipeline
```

```
# base Docker image that we will build on
FROM python:3.14.3-slim
# set up our image by installing prerequisites; pandas in this case
RUN pip install pandas pyarrow
# set up the working directory inside the container
WORKDIR /code
# copy the script to the container. 1st name is source file, 2nd is destination
COPY pipeline.py .
# define what to do first when the container runs
# in this example, we will just run the script
ENTRYPOINT ["python", "pipeline.py"]
```

```
docker run -it --rm test:pipeline Sul Sudeste
```

Rodar PostgreSQL com Docker

```
docker run -it --rm `
  -e POSTGRES_USER=root `
  -e POSTGRES_PASSWORD=root `
  -e POSTGRES_DB=Microdados `
  -v pgdata:/var/lib/postgresql `
  -p 5432:5432 `
```

```
  postgres:18
```

```
uv add --dev pgcli
```

```
uv run pgcli -h localhost -p 5433 -u root -d Microdados
```

```
-- List tables
\dt

-- Create a test table
CREATE TABLE test (id INTEGER, name VARCHAR(50));

-- Insert data
INSERT INTO test VALUES (1, 'Hello Docker');

-- Query data
SELECT * FROM test;

-- Exit
\q
```

```
uv add --dev jupyter
```

```
uv run jupyter notebook
```

```
uv add sqlalchemy "psycopg[binary,pool]"

OU no jupyter

!uv add sqlalchemy "psycopg[binary,pool]"
```

Database Connection

```
from sqlalchemy import create_engine
engine = create_engine('postgresql+psycopg://root:root@localhost:5432/Microdados')
```

DDL Schema

```
print(pd.io.sql.get_schema(df, name='Microdados', con=engine))
```

Output:

```
CREATE TABLE "microdados_cursos" (
 "NU_ANO_CENSO"        BIGINT,
 "NO_REGIAO"        TEXT,
 "NO_UF"        TEXT,
 "SG_UF"        TEXT,
 "NO_MUNICIPIO"        TEXT,
 "CO_MUNICIPIO"        BIGINT,
 "CO_IES"        BIGINT,
 "NO_CURSO"        TEXT,
 "NO_CINE_ROTULO"        TEXT,
 "NO_CINE_AREA_GERAL"        TEXT,
 "CO_CINE_AREA_ESPECIFICA"        BIGINT,
 "NO_CINE_AREA_ESPECIFICA"        TEXT,
 "CO_CINE_AREA_DETALHADA"        BIGINT,
 "NO_CINE_AREA_DETALHADA"        TEXT,
 "TP_GRAU_ACADEMICO"        BIGINT,
 "IN_GRATUITO"        BIGINT,
 "TP_MODALIDADE_ENSINO"        BIGINT,
 "TP_NIVEL_ACADEMICO"        BIGINT,
 "QT_CURSO"        BIGINT,
 "QT_VG_TOTAL"        BIGINT,
 "QT_VG_TOTAL_DIURNO"        BIGINT,
 "QT_VG_TOTAL_NOTURNO"        BIGINT,
 "QT_VG_TOTAL_EAD"        BIGINT,
 "QT_ING"        BIGINT,
 "QT_ING_FEM"        BIGINT,
 "QT_ING_MASC"        BIGINT,
 "QT_ING_DIURNO"        BIGINT,
 "QT_ING_NOTURNO"        BIGINT,
 "QT_ING_OUTRO_TIPO_SELECAO"        BIGINT,
 "QT_ING_PROC_SELETIVO"        BIGINT,
 "QT_ING_0_17"        BIGINT,
 "QT_ING_18_24"        BIGINT,
 "QT_ING_25_29"        BIGINT,
 "QT_ING_30_34"        BIGINT,
 "QT_ING_35_39"        BIGINT,
 "QT_ING_40_49"        BIGINT,
 "QT_ING_50_59"        BIGINT,
 "QT_ING_60_MAIS"        BIGINT,
 "QT_ING_BRANCA"        BIGINT,
 "QT_ING_PRETA"        BIGINT,
 "QT_ING_PARDA"        BIGINT,
 "QT_ING_AMARELA"        BIGINT,
 "QT_ING_INDIGENA"        BIGINT,
 "QT_ING_CORND"        BIGINT
)
```

Criar tabela

```
df.head(n=0).to_sql(name='microdados_cursos', con=engine, if_exists='replace')
```

```
df_iter = pd.read_csv(
    file, sep=';', encoding='utf-8'
    iterator=True,
    chunksize=10000
)
```

Iterar Chunks

```
for df_chunk in df_iter:
    print(len(df_chunk))
```

```
uv add tdqm


NÃo RODAR ASSIM. Rodar com o tqdm
df_chunk.to_sql(name='microdados_cursos', con=engine, if_exists='append')
```

```
from tqdm.auto import tqdm

first = True

for df_chunk in df_iter:
    if first:
        df_chunk.head(0).to_sql(
            name="microdados_cursos",
            con=engine,
            if_exists="replace"
        )
        first = False
        print("Tabela criada")

    df_chunk.to_sql(
        name="microdados_cursos",
        con=engine,
        if_exists="append"
    )
    print("Inseridos:", len(df_chunk))
```

Adicionar status e progresso

```
uv add tqdm
```

```
from tqdm.auto import tqdm

for df_chunk in tqdm(df_iter):
    ...
```

```
uv run pgcli -h localhost -p 5432 -u root -d Microdados
```

```
FROM python:3.14.3-slim

COPY --from=ghcr.io/astral-sh/uv:latest /uv /bin/

WORKDIR /app


ENV PATH="/app/.venv/bin:$PATH"

# Copy dependency files first (better layer caching)
COPY "pyproject.toml" "uv.lock" ".python-version" ./
# Install dependencies from lock file (ensures reproducible builds)
RUN uv sync --locked

COPY codigo_ingestao.py .

ENTRYPOINT ["python", "codigo_ingestao.py"]
```